

Accordo su chiavi

(key agreement)

Alfredo De Santis

Dipartimento di Informatica
Università di Salerno

ads@dia.unisa.it

<http://www.dia.unisa.it/professori/ads>

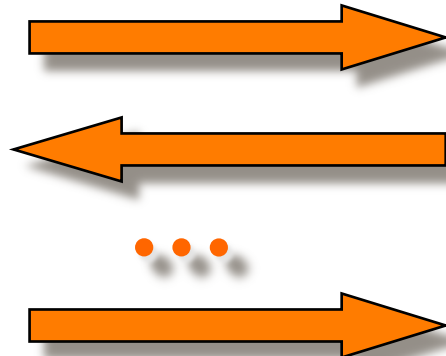
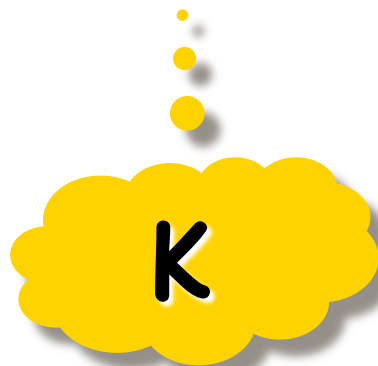


Marzo 2015

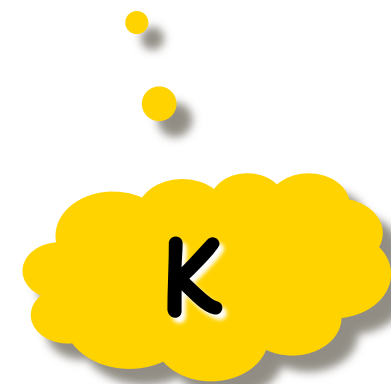
Accordo su una chiave



Alice



Bob



Accordo su chiavi: indice

➤ Diffie-Hellman

- Basato sull'intrattabilità del problema del logaritmo discreto

➤ Puzzle di Merkle

- Non basato su alcuna assunzione computazionale

Diffie-Hellman [1976]

Whitfield Diffie
Martin Hellman

Stanford University

W. Diffie and M.E. Hellman,
New directions in cryptography,
IEEE Trans. on Inform. Theory 22
(1976), 644-654.



Diffie-Hellman [1976]

primo p , generatore g di $\mathbb{Z}_p^*$



Alice



Bob



Generatori di Z_p^*

g è generatore di Z_p^* se $\{ g^i \mid 1 \leq i \leq p-1 \} = Z_p^*$

Esempio:

$g = 2$ è un
generatore
di Z_{11}^*

$$\left\{ \begin{array}{ll} 2^{10} = 1024 & = 1 \bmod 11 \\ 2^1 & = 2 \bmod 11 \\ 2^8 = 256 & = 3 \bmod 11 \\ 2^2 & = 4 \bmod 11 \\ 2^4 = 16 & = 5 \bmod 11 \\ 2^9 = 512 & = 6 \bmod 11 \\ 2^7 = 128 & = 7 \bmod 11 \\ 2^3 & = 8 \bmod 11 \\ 2^6 = 64 & = 9 \bmod 11 \\ 2^5 = 32 & = 10 \bmod 11 \end{array} \right.$$

Generatori di Z_n^*

- Ordine di $\alpha \in Z_n^*$ = il più piccolo intero positivo r tale che $\alpha^r = 1 \pmod n$
- α è generatore di Z_n^* se ha ordine $\phi(n)$

Teorema di Eulero
 $x \in Z_n^* \Rightarrow x^{\phi(n)} = 1 \pmod n$

Generatori di Z_n^*

- Ordine di $\alpha \in Z_n^*$ = il più piccolo intero positivo r tale che $\alpha^r = 1 \pmod n$
- α è generatore di Z_n^* se ha ordine $\phi(n)$

Teorema di Eulero
 $x \in Z_n^* \Rightarrow x^{\phi(n)} = 1 \pmod n$

- Z_n^* ha un generatore $\Leftrightarrow n = 2, 4, p^k, 2p^k$, con p primo e $k \geq 1$
 - Se p è primo, allora Z_p^* ha un generatore

Potenze in $\mathbb{Z}_{19}^*$

[illegible]

Generatori di Z_n^*

- Ordine di $\alpha \in Z_n^*$ = il più piccolo intero positivo r tale che $\alpha^r = 1 \pmod n$
- a è generatore di Z_n^* se ha ordine $\phi(n)$
- Se α è un generatore di Z_n^* , allora
 - $Z_n^* = \{\alpha^i \pmod n \mid 0 \leq i \leq \phi(n)-1\}$
 - $b = \alpha^i \pmod n$ è un generatore di $Z_n^* \iff \gcd(i, \phi(n))=1$

Esempio:

2 è un generatore in Z_{19}^*

- $3=2^{13} \pmod{19}$ è un generatore perchè $\gcd(13,6)=1$
- $13=2^5 \pmod{19}$ è un generatore perchè $\gcd(5,6)=1$
- $5=2^{16} \pmod{19}$ non è un generatore perchè $\gcd(16,6)=2$

Potenze in $\mathbb{Z}_{19}^*$

[illegible]

Generatori di Z_n^*

- Il numero di generatori di Z_n^* è $\phi(\phi(n))$
 - Se p è primo, il numero di generatori di Z_p^* è $\phi(p-1)$

Diffie-Hellman [1976]

primo p , generatore g di Z_p^*

scelgo $x \in Z_p$



Alice

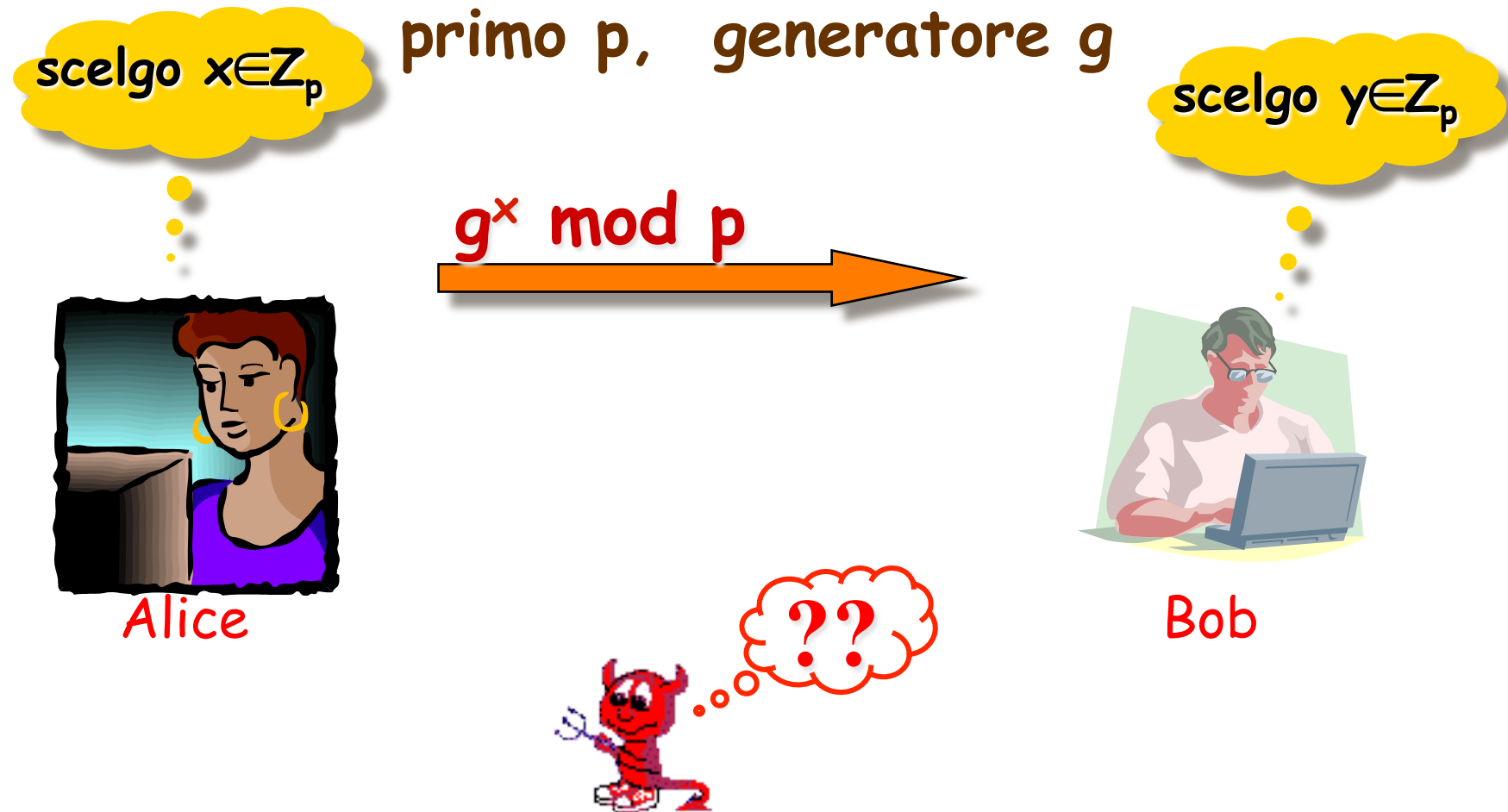
scelgo $y \in Z_p$



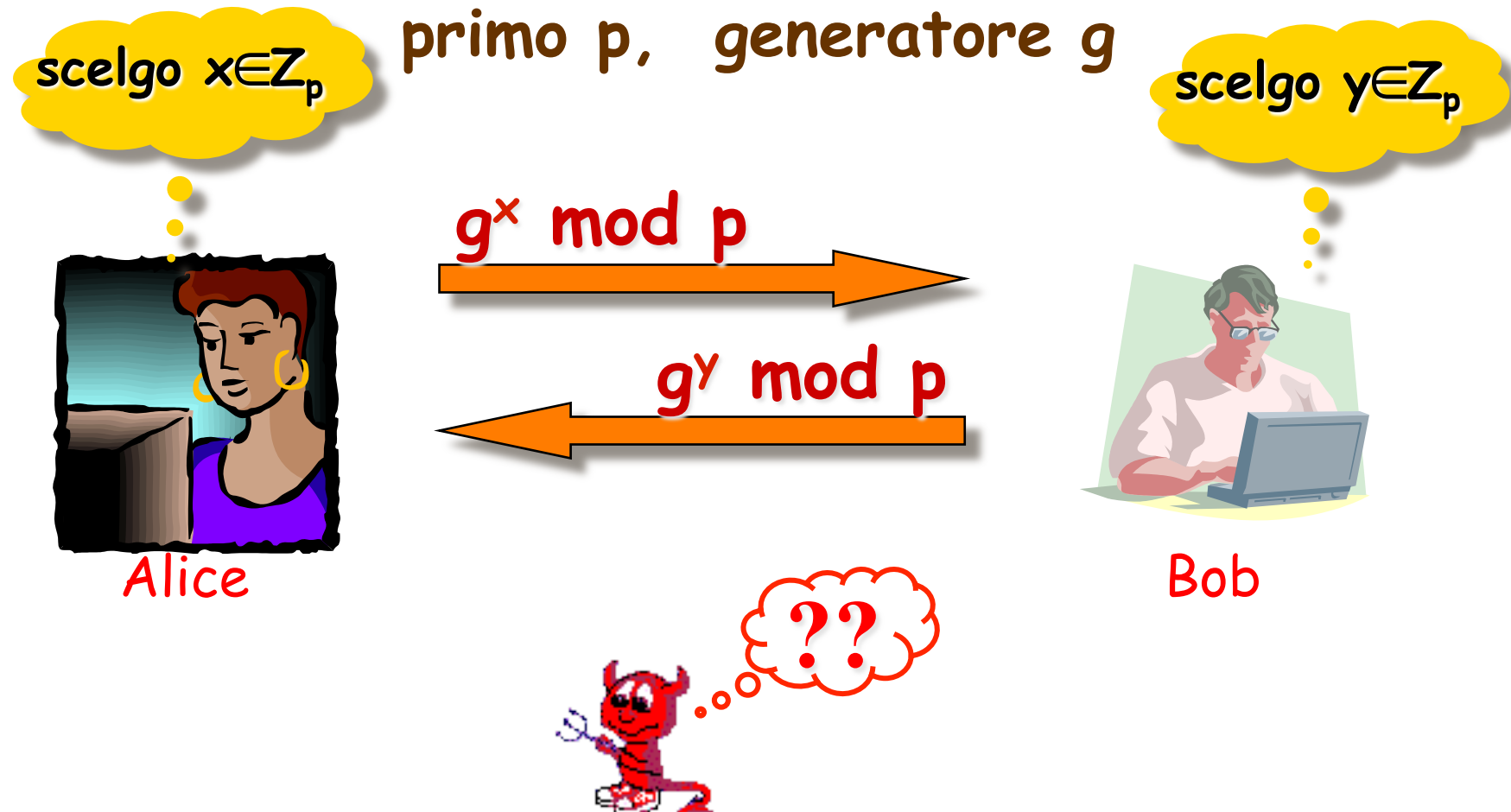
Bob



Diffie-Hellman [1976]



Diffie-Hellman [1976]



Diffie-Hellman [1976]

scelgo $x \in \mathbb{Z}_p$

primo p , generatore g

scelgo $y \in \mathbb{Z}_p$



Alice

$g^x \bmod p$

$g^y \bmod p$



Bob

$$K = g^{xy} \bmod p \\ = (g^y)^x \bmod p$$



??

$$K = g^{xy} \bmod p \\ = (g^x)^y \bmod p$$

Diffie-Hellman: "piccolo" esempio

primo 11, generatore 2

scelgo $x=3$



Alice

$$8 = 2^3 \bmod 11$$



$$5 = 2^4 \bmod 11$$

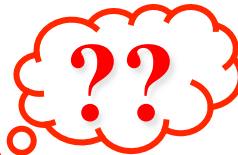


scelgo $y=4$



Bob

$$K=4=(2^4)^3 \bmod 11$$



$$K=4=(2^3)^4 \bmod 11$$

Diffie-Hellman: esempio

scelgo
 $x=3578$



Alice

primo 25307, generatore 2

$$6113 = 2^{3578} \bmod 25307$$



$$7984 = 2^{19956} \bmod 25307$$

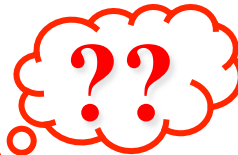


scelgo
 $y=19956$



Bob

$$K=3694=7984^{3578}$$



$$K=3694=6113^{19956}$$

Logaritmo discreto

La sicurezza di molte tecniche crittografiche si basa sulla **intrattabilità** del **logaritmo discreto**:

- Crittosistema ElGamal
- Accordo su chiavi Diffie-Hellman
- Firme digitali DSS

Dati a, n, b calcolare x tale che $a^x = b \pmod n$

Esempio: $3^x = 7 \pmod{13}$

soluzione $x = 6$

Logaritmo discreto: Complessità algoritmi

Dati a, n, b calcolare x tale che $a^x = b \pmod n$

Vari algoritmi:

- Trial multiplication
- Baby-step giant-step
- Pollard's rho algorithm
- Pohlig-Hellman algorithm
- Index Calculus

Logaritmo discreto: Complessità algoritmi

Dati a, n, b calcolare x tale che $a^x = b \pmod n$

Se n è primo, i migliori algoritmi hanno complessità

$$L_n[a, c] = O(e^{(c+o(1))} (\ln n)^a (\ln \ln n)^{1-a})$$

con $c > 0$ ed $0 < a < 1$

Miglior algoritmo: **Number field sieve**

tempo medio euristico $L_n[1/3, 1.923]$

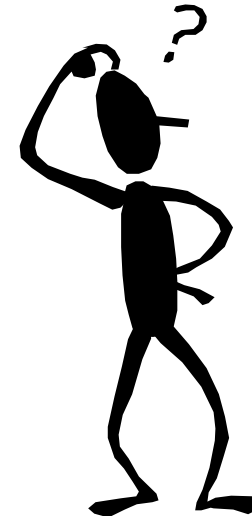

$$\left(\frac{64}{9}\right)^{1/3} = 1,922999$$

Logaritmo discreto: record computazioni

- Primo, 130-digit, 431-bit: giugno 2005, A. Joux e R. Lercier
 - 3 settimane, 1.15 GHz 16-processor HP AlphaServer GS1280
- Primo, 160-digit, 530-bit: febbraio 2007, T. Kleinjung
 - Vari pc ed un cluster
- $GF(2^{613})$, settembre 2005, A. Joux e R. Lercier
 - 17 giorni, 4 nodi 1.3GHz 16-processor di un Itanium 2-based Bull computer Teranova
- $GF(2^{809})$, aprile 2013, R. Barbulescu, C. Bouvier, ...
- $GF(2^{18 \cdot 513})$, 9234-bit, gen 2014, R. Granger, T. Kleinjung, J. Zumbrägel
- $GF(3^{6 \cdot 71})$, 676-bit, 2010, T. Hayashi
- $GF(3^{6 \cdot 137})$, 1303-bit, 27 gen 2014, F. Rodríguez-Henríquez

Problema di Diffie-Hellman

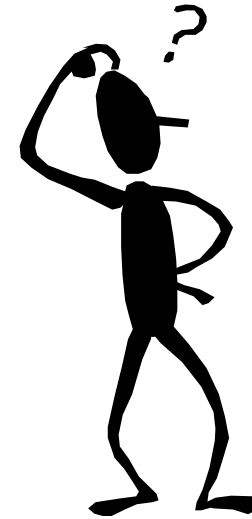
Input: primo p , generatore g ,
 $g^x \bmod p$, $g^y \bmod p$
Calcolare: $g^{xy} \bmod p$



Il miglior algoritmo conosciuto calcola prima
il logaritmo discreto $x \leftarrow \log_{g,p}(g^x \bmod p)$

Problema di Diffie-Hellman

Input: primo p , generatore g ,
 $g^x \bmod p$, $g^y \bmod p$
Calcolare: $g^{xy} \bmod p$



Il miglior algoritmo conosciuto calcola prima
il logaritmo discreto $x \leftarrow \log_{g,p}(g^x \bmod p)$

... ma non si sa se sono equivalenti!

Scelta dei parametri

➤ Come scegliere p e g ?



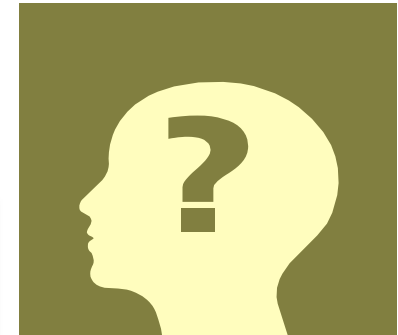
Scelta di un generatore



Scegli_Generatore_Naive (p)

1. Scegli a caso g in $\mathbb{Z}_p^*$
2. **If** $\{g^i | 1 \leq i \leq p-1\} = \mathbb{Z}_p^*$
 then trovato
 else goto 1.

Scelta di un generatore



Scegli_Generatore_Naive (p)

1. Scegli a caso g in $\mathbb{Z}_p^*$
2. **If** $\{g^i | 1 \leq i \leq p-1\} = \mathbb{Z}_p^*$
 then trovato
 else goto 1.

$$\{g^i | 1 \leq i \leq p-1\} = \mathbb{Z}_p^* ?$$



L'unico algoritmo
efficiente necessita
dei fattori primi di
 $p-1$

Scelta di un generatore

p primo, $p-1 = p_1^{e_1} p_2^{e_2} \dots p_k^{e_k}$

g è un generatore di Z_p^* $\Leftrightarrow$

$$\begin{cases} g^{(p-1)/p_1} \not\equiv 1 \pmod{p} \\ \dots \\ g^{(p-1)/p_k} \not\equiv 1 \pmod{p} \end{cases}$$

Scelta di un generatore

$$\begin{array}{l} p \text{ primo, } p-1 = p_1^{e_1} p_2^{e_2} \dots p_k^{e_k} \\ g \text{ è un generatore di } Z_p^* \Leftrightarrow \end{array} \left\{ \begin{array}{l} g^{(p-1)/p_1} \not\equiv 1 \pmod{p} \\ \dots \\ g^{(p-1)/p_k} \not\equiv 1 \pmod{p} \end{array} \right.$$

□ 11 primo, $p-1 = 10 = 2 \cdot 5$

Esempio

□ 2 è un generatore di Z_{11}^* perché

$$2^{(11-1)/2} = 2^5 = 10 \not\equiv 1 \pmod{11}$$

$$2^{(11-1)/5} = 2^2 = 4 \not\equiv 1 \pmod{11}$$

Scelta di un generatore

$$\begin{array}{l} p \text{ primo, } p-1 = p_1^{e_1} p_2^{e_2} \dots p_k^{e_k} \\ g \text{ è un generatore di } Z_p^* \Leftrightarrow \end{array} \left\{ \begin{array}{l} g^{(p-1)/p_1} \not\equiv 1 \pmod{p} \\ \dots \\ g^{(p-1)/p_k} \not\equiv 1 \pmod{p} \end{array} \right.$$

□ 11 primo, $p-1 = 10 = 2 \cdot 5$

Esempio

□ 3 non è un generatore di Z_{11}^* perché

$$3^{(11-1)/2} = 3^5 = 243 \equiv 1 \pmod{11}$$

$$3^{(11-1)/5} = 3^2 = 9 \not\equiv 1 \pmod{11}$$

Scelta di un generatore

$$\begin{array}{l} p \text{ primo, } p-1 = p_1^{e_1} p_2^{e_2} \dots p_k^{e_k} \\ g \text{ è un generatore di } Z_p^* \Leftrightarrow \left\{ \begin{array}{l} g^{(p-1)/p_1} \not\equiv 1 \pmod{p} \\ \dots \\ g^{(p-1)/p_k} \not\equiv 1 \pmod{p} \end{array} \right. \end{array}$$

Scegli_generatore ($p, (p_1, e_1, p_2, e_2, \dots, p_k, e_k)$)

1. $g \leftarrow$ elemento scelto a caso in Z_p^*
2. if $(g^{(p-1)/p_1} \not\equiv 1 \pmod{p} \text{ and } \dots \text{ and } g^{(p-1)/p_k} \not\equiv 1 \pmod{p})$
 then esci **trovato!** 
 else go to 1.

Probabilità successo singola iterazione

- Numero di generatori modulo un primo p è

$$\phi(\phi(p)) = \phi(p-1)$$

$$> (p-1) / (6 \cdot \ln \ln(p-1))$$

per ogni intero $n \geq 5$,
 $\phi(n) > n / (6 \ln \ln n)$

Probabilità successo singola iterazione

- Numero di generatori modulo un primo p è

$$\phi(\phi(p)) = \phi(p-1)$$

$$> (p-1) / (6 \cdot \ln \ln(p-1))$$

per ogni intero $n \geq 5$,
 $\phi(n) > n / (6 \ln \ln n)$

- Probabilità che un elemento a caso in Z_p^* sia generatore

$$= \frac{\phi(\phi(p))}{\phi(p)} > \frac{p-1}{\phi(p) \cdot 6 \ln \ln(p-1)} = \frac{1}{6 \cdot \ln \ln(p-1)}$$

Analisi di Scegli_generatore

Numero medio di iterazioni $< 6 \cdot \ln \ln(p - 1)$

512 bit	$6 \cdot \ln \ln(2^{512}) \approx 35,23$
1024 bit	$6 \cdot \ln \ln(2^{1024}) \approx 39,38$
2048 bit	$6 \cdot \ln \ln(2^{2048}) \approx 43,54$

Generazione chiavi Diffie-Hellman

1. Scegli a caso 2 numeri primi p_1 p_2
2. $p \leftarrow 1 + 2p_1p_2$
3. Se p non è primo, go to 1.
4. $g \leftarrow \text{Scegli_generatore}(p, (2, 1, p_1, 1, p_2, 1))$

Accordo su chiave Diffie-Hellman: sicurezza

- E' sicuro contro attaccanti passivi



- Non è sicuro contro attacchi
man-in-the-middle



Diffie-Hellman

attacco man-in-the-middle

primo p , generatore g di $\mathbb{Z}_p^*$

scelgo $x \in \mathbb{Z}_p$



Alice

$g^x \bmod p$



scelgo $z \in \mathbb{Z}_p$



scelgo $y \in \mathbb{Z}_p$



Bob

$g^z \bmod p$



$g^y \bmod p$



$g^z \bmod p$



$$K_A = g^{xz} \bmod p \\ = (g^z)^x \bmod p$$

$$K_A = g^{xz} \bmod p \\ = (g^x)^z \bmod p \\ K_B = g^{zy} \bmod p \\ = (g^y)^z \bmod p$$

$$K_B = g^{zy} \bmod p \\ = (g^z)^y \bmod p$$

Accordo su chiave Diffie-Hellman: sicurezza

- E' sicuro contro attaccanti passivi



- Non è sicuro contro attacchi *man-in-the-middle*
- Occorre **autenticazione**

Crittografia a chiave pubblica con Diffie-Hellman

- Possiamo usare Diffie-Hellman come cifrario asimmetrico?



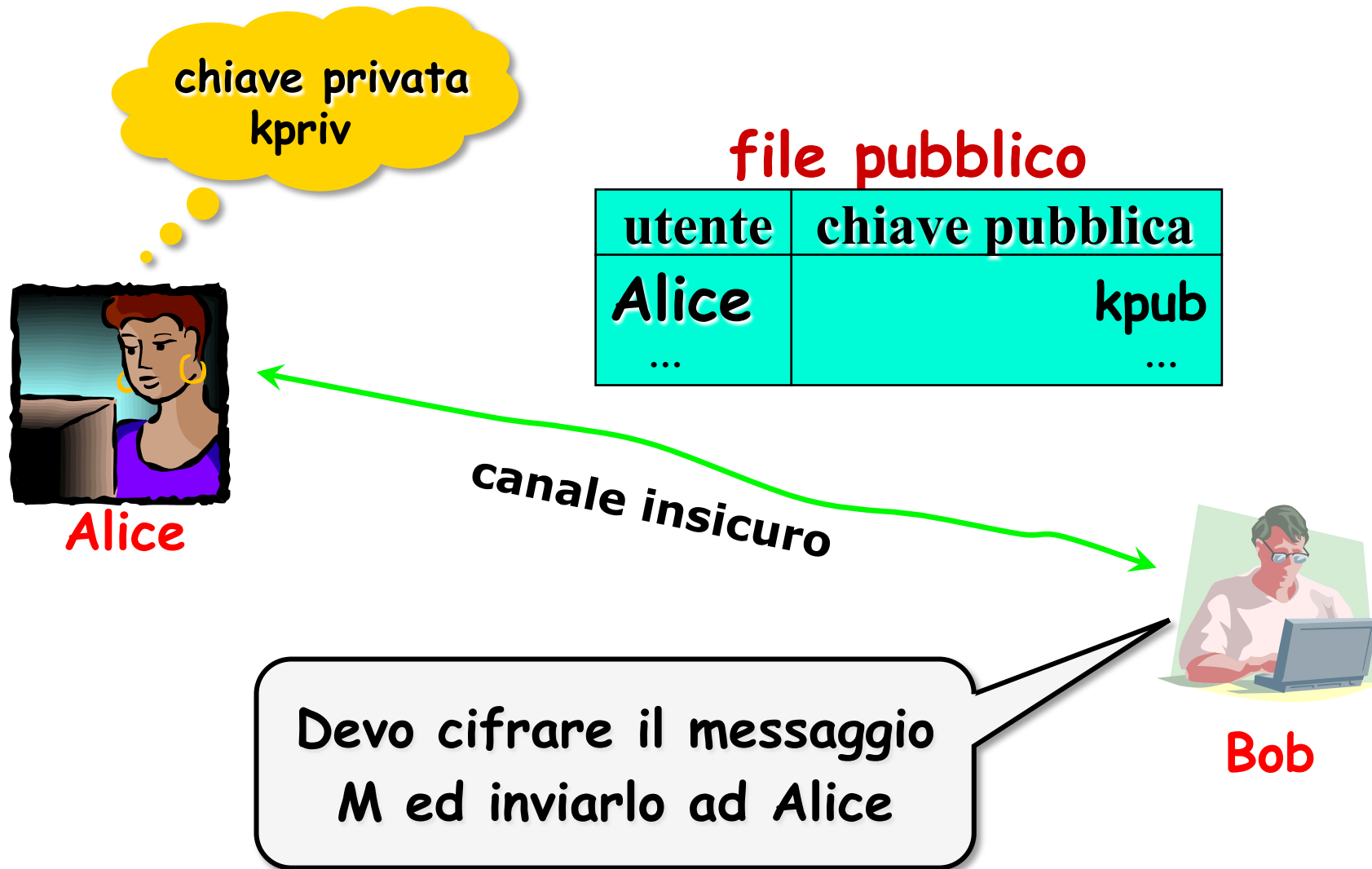
Cifrari asimmetrici



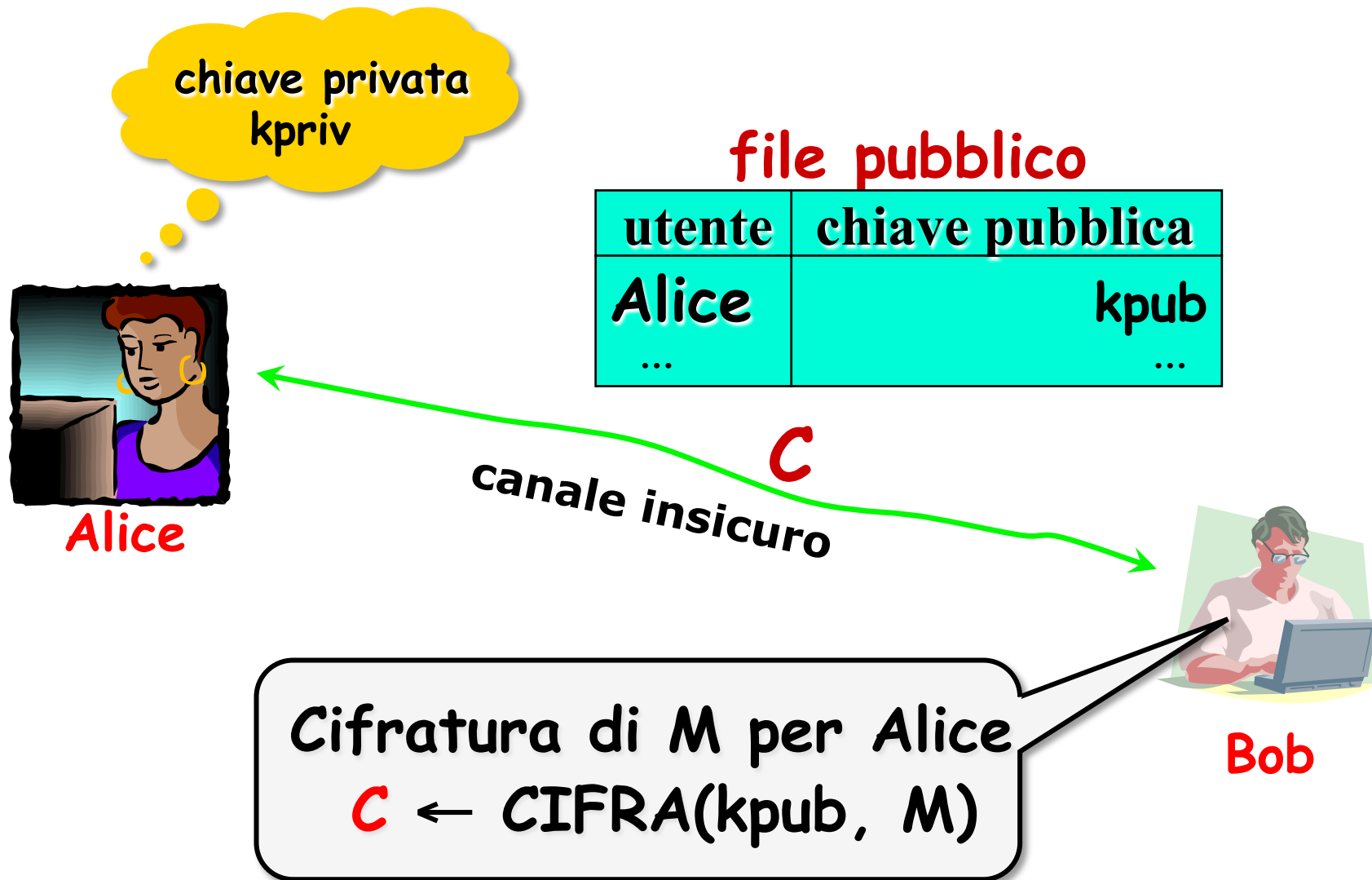
file pubblico

utente	chiave pubblica
Alice	kpub
...	...

Cifratura



Cifratura



Decifratura

Devo decifrare il
messaggio cifrato **C**

file pubblico

utente	chiave pubblica
Alice	kpub
...	...



Alice



Decifratura

chiave privata
 k_{priv}

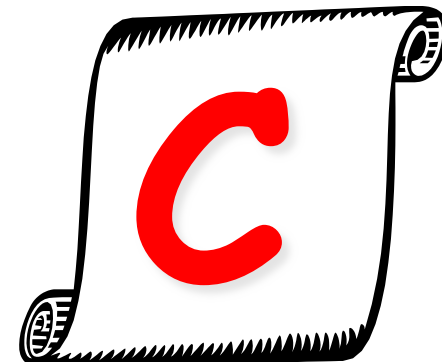
file pubblico

utente	chiave pubblica
Alice	k_{pub}
...	...



Alice

Decifratura di C
 $M \leftarrow \text{DECIFRA}(k_{priv}, C)$



Chiavi DH

chiave privata
(p, g, a)

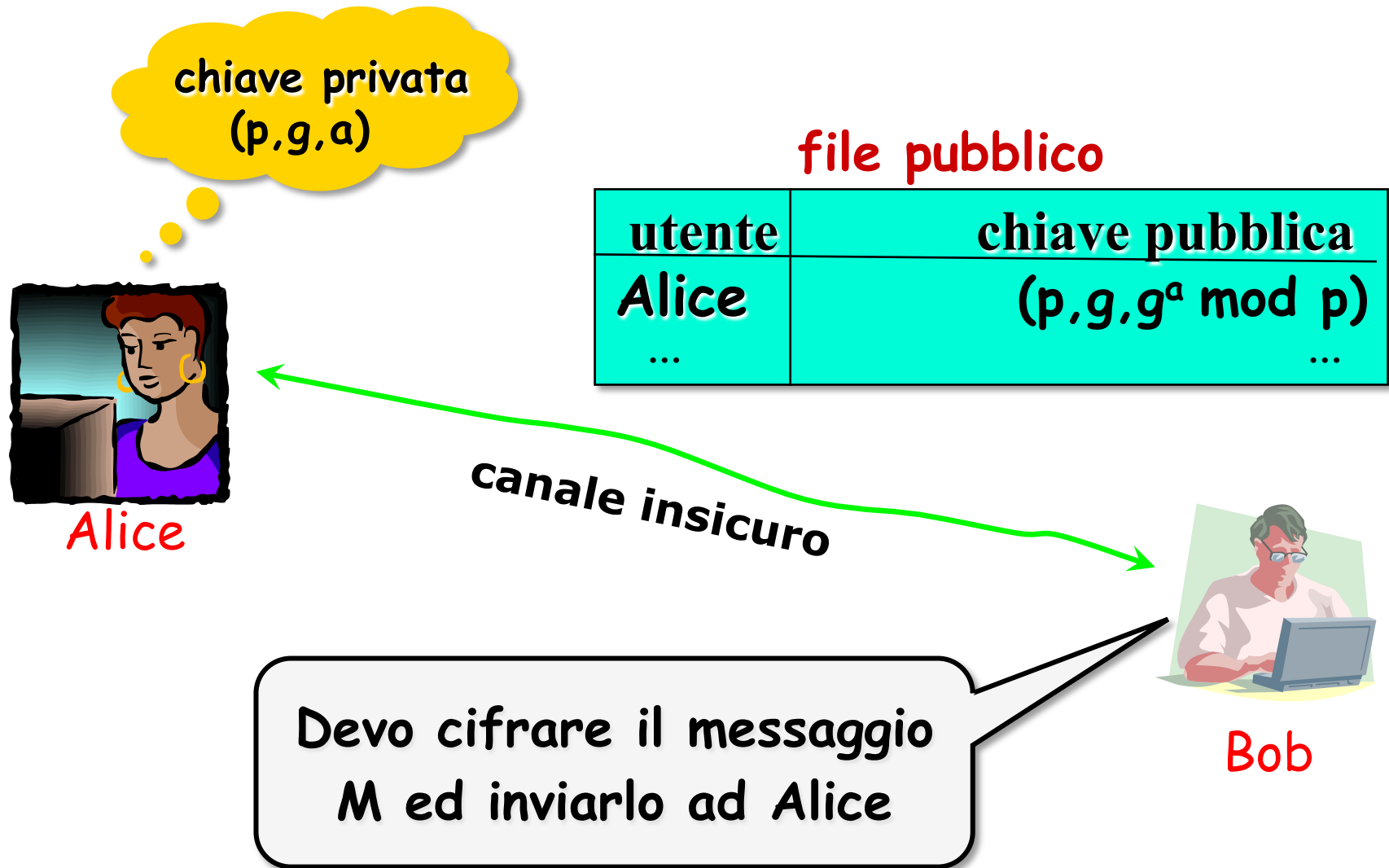


Alice

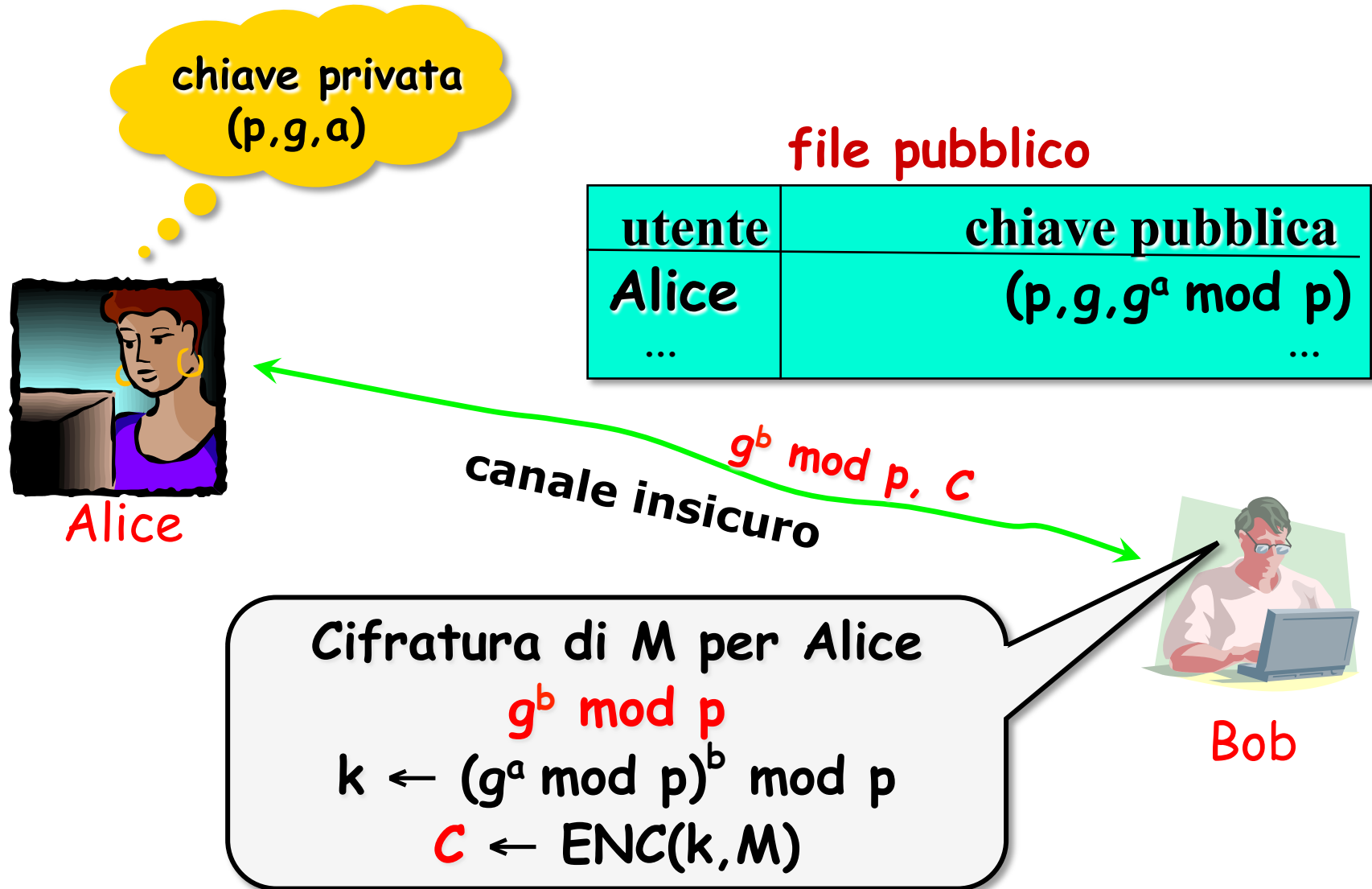
file pubblico

utente	chiave pubblica
Alice	$(p, g, g^a \bmod p)$
...	...

Cifratura DH



Cifratura DH



Decifratura DH

Devo decifrare il
messaggio cifrato
 $(g^b \bmod p, C)$



Alice

file pubblico

utente	chiave pubblica
Alice	$(p, g, g^a \bmod p)$
...	...

$g^b \bmod p, C$



Cifratura DH

chiave privata
(p, g, a)



Alice

file pubblico

utente	chiave pubblica
Alice	$(p, g, g^a \bmod p)$
...	...

Decifratura di $g^b \bmod p, C$

$$k \leftarrow (g^b \bmod p)^a \bmod p$$

$$M \leftarrow \text{DEC}(k, C)$$

Crittografia a chiave pubblica

RSA e Diffie-Hellman

- RSA è il cifrario asimmetrico più diffuso
- Motivazioni:
 - Storiche
 - Commerciali
 - Verisign fondata come spin-off di RSA Security nel 1995
 - Algoritmi di firma digitale



Accordo su chiavi: indice

➤ Diffie-Hellman

- Basato sull'intrattabilità del problema del logaritmo discreto

➤ Puzzle di Merkle

- Non basato su alcuna assunzione computazionale

Schema di Merkle

Ralph C. Merkle

Corso di Computer Security,
UC Berkeley, autunno 1974:
proposta per un progetto, rifiutata
dal docente

<http://merkle.com/1974/>



Ralph C. Merkle

"Secure Communications over Insecure Channels"

Communications of the ACM, vol. 21, n. 4, April 1978, pp. 294-299

Schema di Merkle

- Non basato su assunzioni computazionali
- Alice genera n chiavi distinte e "nasconde" ogni chiave in un **puzzle**
 - Il puzzle contiene informazioni per il calcolo della chiave
 - La soluzione di **un puzzle** richiede un tempo ragionevole
 - La soluzione di **tutti i puzzle** richiede un tempo elevato

Puzzle di Merkle

- Puzzle la cui soluzione richiede t operazioni

- Esempio:

Puzzle (x, ID, S)

Scegli una chiave k di 56 bit

Computa $y \leftarrow \text{CBC-DES}_k(x, ID, S)$

return (y , primi 20 bit di k)

- x è la soluzione del puzzle

- Richiede 2^{35} operazioni in media

- ID è l'identificativo del puzzle

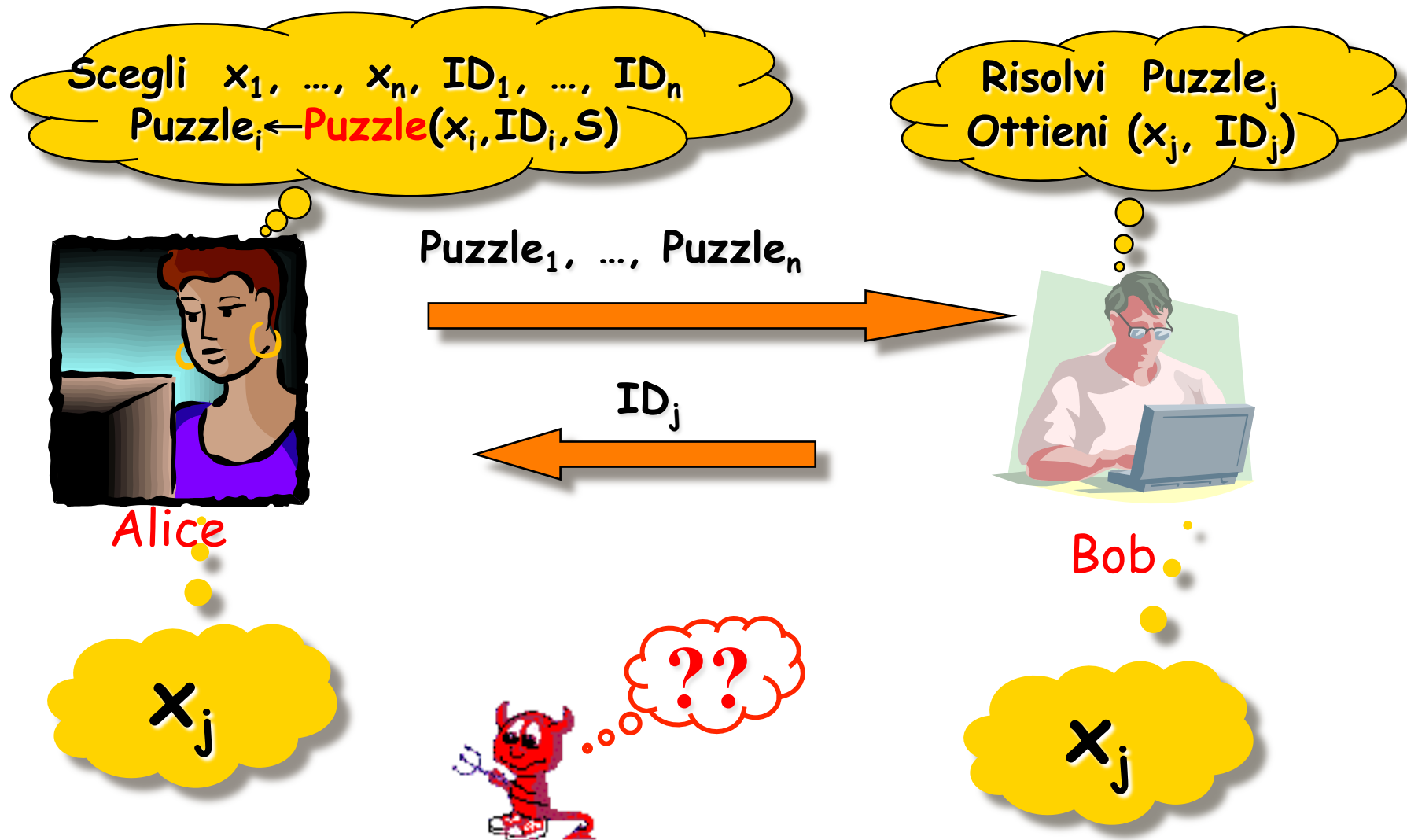
- Unico per ciascun puzzle

- S è un valore noto

- Serve per garantire l'unicità della soluzione del puzzle

- Esempio: 32 bit nulli

Puzzle di Merkle



Puzzle di Merkle

Computazioni di  :

- Costruzione di n puzzle

tempo $\theta(n)$

Computazioni di  :

- Risoluzione di un puzzle

tempo $\theta(t)$

Computazioni di  :

- Risoluzione di $n/2$ puzzle in media

tempo $\theta(t \cdot n)$

Puzzle di Merkle

Computazioni di



:

- Costruzione di n puzzle

Se $n = \theta(t)$

tempo $\theta(n)$

Computazioni di



- Risoluzione di un puzzle

tempo $\theta(n)$

Computazioni di



- Risoluzione di $n/2$ puzzle in media

tempo $\theta(n^2)$

Puzzle di Merkle

problema di sicurezza

- Implementazione poco accorta potrebbe rendere insicuro lo schema

- Esempio:

Puzzle (x , ID, S)

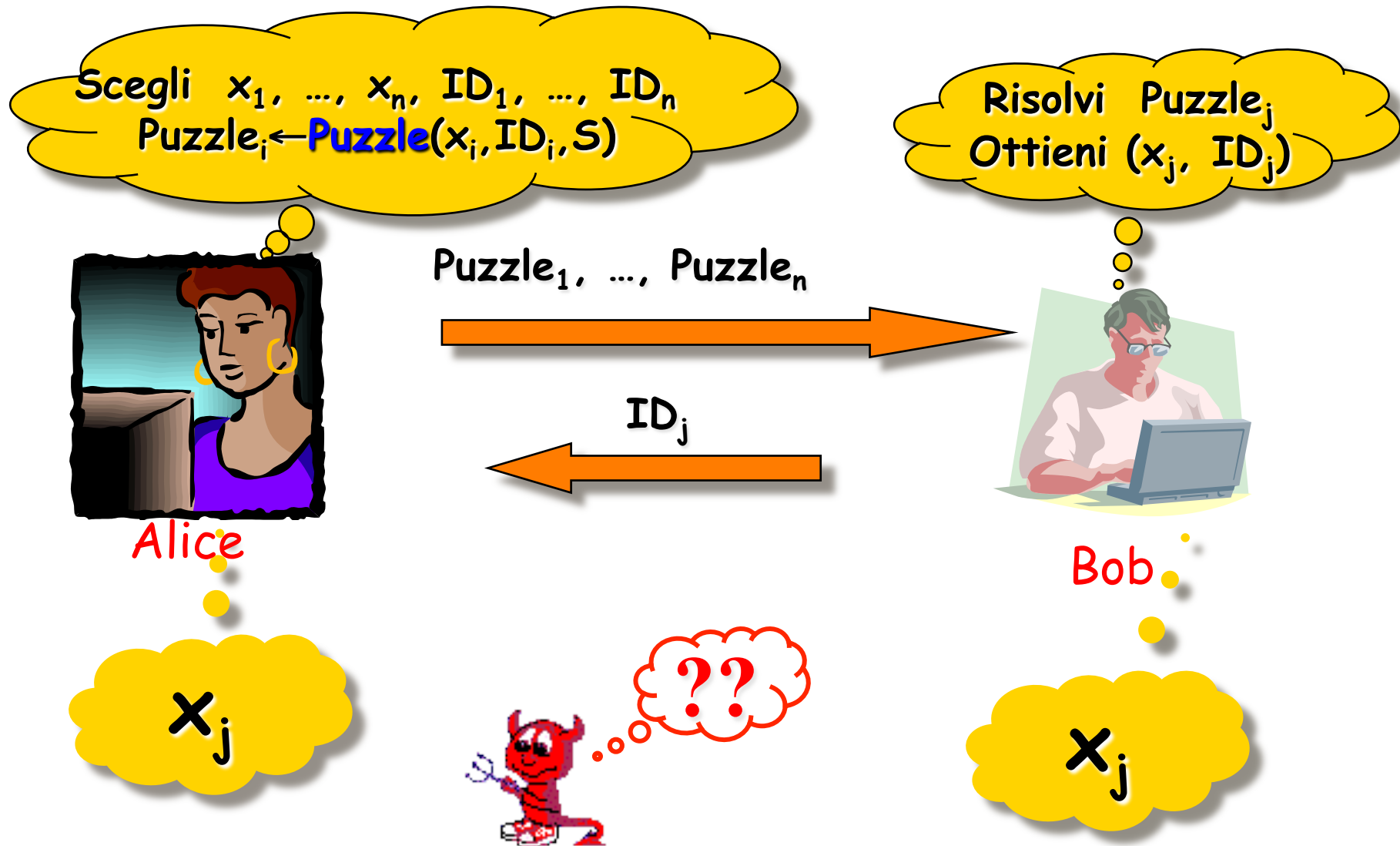
Scegli una chiave k di 56 bit

Computa $y \leftarrow \text{DES}_k(x), \text{DES}_k(\text{ID}), \text{DES}_k(S)$

return (y , primi 20 bit di k)

- x è la soluzione del puzzle
 - Richiede 2^{35} operazioni in media
- ID è l'identificativo del puzzle
 - Unico per ciascun puzzle
- S è un valore noto
 - Serve per garantire l'unicità della soluzione del puzzle
 - Esempio: 32 bit nulli

Puzzle di Merkle



Puzzle di Merkle

problema di sicurezza

- Implementazione poco accorta potrebbe rendere insicuro lo schema

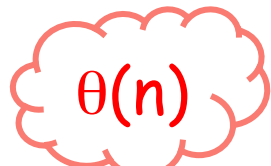
- Esempio:

Puzzle (x , ID, S)

Scegli una chiave k di 56 bit

Computa $y \leftarrow \text{DES}_k(x), \text{DES}_k(\text{ID}), \text{DES}_k(S)$

return (y , primi 20 bit di k)



$\theta(n)$

- Dopo aver visto ID_j
- Cifra $\text{DES}_k(\text{ID}_j)$ con tutte le possibili chiavi k
- Correla le cifrature con tutti i Puzzle visti
- Trova il Puzzle con ID_j
- Decifra il corrispondente $\text{DES}_k(x)$

Puzzle di Merkle

problema di sicurezza

- Implementazione poco accorta potrebbe rendere insicuro lo schema

- Esempio:

Puzzle (x , ID, S)

Scegli una chiave k di 56 bit

Computa $y \leftarrow \text{DES}_k(x), \text{DES}_k(\text{ID}), \text{DES}_k(S)$

return (y , primi 20 bit di k)

NO

Meglio:

- $\text{DES}_k(x), \text{DES}_k(\text{ID} \oplus x), \text{DES}_k(S \oplus x)$ con $S \neq 0$
- $\text{DES}_k(x), \text{DES}_x(\text{ID}), \text{DES}_x(S)$
- $\text{AES}_k(x, \text{ID}, S)$
 - Bene per x di 64 bit (con ID ed S di 32 bit)
 - Per x di 128 bit si può usare 2 volte lo schema

Bibliografia

- **Cryptography and Network Security**
by W. Stallings (2010)
 - cap. 10
- Tesina di Sicurezza su reti
 - Accordo su chiavi

Domande?

